Endpoint Reporting & Dashboards for Intune and Configuration Manager

Implementation Solution Guide

Version 1.0.0 · 2026-06-28

Contents

Executive Summary — Endpoint Reporting & Dashboards for Intune and Configuration Manager 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Endpoint Reporting & Dashboards for Intune and Configuration Manager 5

What this solution is . . . 5

How it works . . . 5

The endpoint KPI set (what "good" looks like) . . . 5

Why a reporting layer (not just the consoles) . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Endpoint Reporting & Dashboards 7

2.1 Licensing . . . 7

2.2 Devices reporting into Intune . . . 7

2.3 Reporting permissions (read-only Graph scopes) . . . 7

2.4 Microsoft Graph export app registration . . . 7

2.5 Reporting-scope groups . . . 8

2.6 A reporting destination . . . 8

2.7 (If applicable) Configuration Manager reporting point . . . 8

2.8 Defined KPI set agreed with stakeholders . . . 8

Checklist . . . 8

3. Implementation Guide — Endpoint Reporting & Dashboards 9

Step 1 — Confirm readiness . . . 9

Step 2 — Define the KPI set . . . 9

Step 3 — Create the reporting-scope groups . . . 9

Step 4 — Locate and confirm the built-in Intune reports . . . 10

Step 5 — Register the Graph export app . . . 10

Step 6 — Set up the scheduled export (dry run first) . . . 10

Step 7 — (If applicable) Locate the Configuration Manager reports . . . 10

Step 8 — Build the consolidated KPI report . . . 10

Step 9 — Validate the pilot (the gate) . . . 11

Step 10 — Expand and distribute . . . 11
5. Testing Guide — Endpoint Reporting & Dashboards 11
Test 1 — Data sources readable . . . 11
Test 2 — Scope groups correct . . . 11
Test 3 — Export produces files . . . 11
Test 4 — KPI report renders . . . 12
Test 5 — Numbers reconcile (the gate) . . . 12
Test 6 — Configuration Manager devices counted (if applicable) . . . 12
Test 7 — Distribution cycle . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Endpoint Reporting & Dashboards 12
A. Cannot sign in / read a data source . . . 12
B. Numbers don't reconcile (KPI mismatch) . . . 13
C. Export job problems . . . 13
D. Configuration Manager reports . . . 14
E. Scope-group issues . . . 14
F. Diagnostics quick reference . . . 14
G. When to escalate . . . 14
7. Best Practices — Endpoint Reporting & Dashboards 14
8. Deliverables Summary — Endpoint Reporting & Dashboards 15
Documentation . . . 15
PowerShell scripts . . . 15
Named objects (reuse verbatim) . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Endpoint Reporting & Dashboards for Intune and Configuration Manager

When someone asks "how many devices are patched?" or "what's our compliance rate?", the honest answer is too often a scramble across three consoles and a spreadsheet — and two people pulling the same KPI get two different numbers. This solution delivers a **repeatable endpoint reporting layer** across **Microsoft Intune** and **Microsoft Configuration Manager**: a defined KPI set sourced the same way every time, built-in reports, scheduled Microsoft Graph data exports, and a distribution cadence — so the numbers leadership and auditors see are consistent and reproducible.

This is a complete, **deploy-it-today** package, written for an administrator with no prior reporting experience.

What you get

• A plain-language **Solution Overview** of the Intune/Configuration Manager data sources, the endpoint KPI set, and the Graph export model.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite and the named objects to create.

• An **Implementation Guide** of precise, ordered steps: define KPIs, locate the built-in reports, register the Graph export app, build the dashboard, schedule distribution, and validate.

• Five **PowerShell scripts** for reporting-readiness discovery, count reconciliation, a consolidated KPI report, a scheduled Graph export helper, and a reporting-scope group builder.

• **Testing** and **Troubleshooting** guides for a confident, audit-ready rollout.

What this delivers

A **defined endpoint KPI set** — patch compliance %, device compliance %, encryption coverage %, app deployment success %, and device health % — each tied to a single authoritative source, pulled via **Microsoft Graph** `exportJobs`, consolidated into one report, and distributed on a schedule. The fleet view includes **Configuration Manager (SSRS)** devices, not just the Intune slice.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a pilot gate that proves the numbers reconcile against the consoles before broad distribution.

Outcome

Trustworthy, repeatable endpoint reporting — the same numbers every time for leadership and auditors — deployable by a single administrator and validated before it scales.

1. Solution Overview — Endpoint Reporting & Dashboards for Intune and Configuration Manager

What this solution is

This is a **repeatable endpoint reporting layer** that sits on top of the endpoint data you already own in **Microsoft Intune** and **Microsoft Configuration Manager**. Instead of three consoles and a hand-built spreadsheet, you get a defined set of **endpoint KPIs** (patch compliance, device compliance, encryption coverage, app deployment success, device health) sourced the same way every time, plus a scheduled **Microsoft Graph data export** so the numbers leadership and auditors see are consistent and reproducible.

How it works

1. You **define a fixed KPI set** with one authoritative data source per KPI, so the same question always returns the same number.
1. **Intune built-in reports** (`intune.microsoft.com` → **Reports**: Device compliance, Microsoft Defender, Windows updates, etc.) provide the on-screen numbers and ad-hoc views.
1. **Microsoft Graph data export** (`deviceManagement` reports / `exportJobs`) pulls the same report data on a schedule into CSV/JSON for dashboards and archiving — the same payload the portal shows, but automatable.
1. **Configuration Manager reporting** (Configuration Manager console → **Monitoring** → **Reporting**, backed by SQL Server Reporting Services / SSRS) covers any devices still managed by Configuration Manager so the fleet view is complete.
1. **Reporting-scope groups** in Microsoft Entra ID let you target reporting and exports to the right population (pilot first, then the whole fleet).
1. A **distribution schedule** delivers the consolidated KPI report to the people who need it on a recurring cadence.

The endpoint KPI set (what "good" looks like)

KPI	Definition	Primary source
Patch compliance %	Devices on the current/approved quality update ÷ all in-scope devices	Intune **Windows updates** report / Graph export; Configuration Manager software-update SSRS report
Device compliance %	Devices marked **Compliant** ÷ all in-scope devices	Intune **Device compliance** report / Graph export

KPI	Definition	Primary source
Encryption coverage %	Devices with disk encryption (BitLocker/FileVault) on ÷ all in-scope devices	Intune **Encryption report** / Graph export
App deployment success %	App install **Succeeded** ÷ targeted installs	Intune **App install status** report / Graph export
Device health %	Devices checked in recently and not in error/quarantine ÷ all in-scope devices	Intune managed-device + Defender data / Graph export

Why a reporting layer (not just the consoles)

The consoles answer "right now, on this screen." A reporting layer answers "the same way, every time, for everyone" — which is what an audit, a board slide, or a monthly operations review needs. Defining the KPI and its single source removes the disagreement about whose number is right.

Benefits

- **Trust** — one defined source per KPI; the same number every time it is asked.
- **Repeatable** — scheduled Graph exports reproduce the figures without manual console work.
- **Complete** — Intune and Configuration Manager devices both counted.
- **Audit-ready** — exports are archived with a timestamp and scope.
- **Targeted** — reporting-scope groups keep pilot and production numbers separate until validated.

Limitations (be honest with buyers)

- This is a **reporting/visibility** layer over existing data — it does not change patching, compliance, or encryption policy; it measures them.
- Intune reports surface data already collected; a device that has not checked in recently is counted as last-known, not real-time.
- Configuration Manager reporting requires a working **SSRS** reporting point; this solution shows where to read it, not how to stand up Configuration Manager.
- Graph exports return the same data as the portal, so a KPI is only as accurate as the underlying Intune/Configuration Manager data feeding it.
- Dashboards can be built in your tool of choice (Power BI, Excel, etc.); this solution standardizes the **data and KPIs**, not a specific dashboard product.

The standard deployment model

> ***Define KPIs → enable/locate the built-in reports → set up the Graph export app → assemble the dashboard → schedule distribution → validate the numbers reconcile → expand.*** *The repeatable way:* ***Assess → Configure → Pilot → Validate → Expand****, with a pilot gate that proves the numbers reconcile before broad distribution.*

2. Requirements Guide — Endpoint Reporting & Dashboards

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-infrastructure.md`.

Named objects this solution creates (reuse these names verbatim):

- App registration: `Endpoint Reporting Export – Tenant – Prod`
- Reporting-scope group (pilot): `Endpoint Reporting Scope – Pilot – Stage`
- Reporting-scope group (fleet): `Endpoint Reporting Scope – Fleet – Prod`
- KPI report definition: `Endpoint KPI Report – Fleet – Prod`
- Distribution schedule: `Endpoint KPI Distribution – Leadership – Prod`

2.1 Licensing

Description: Intune (Plan 1, included in M365 E3/E5 or Business Premium). Reporting is built in. **Why:** Intune produces the device/compliance/update/encryption reports this solution consumes. **Verify:** M365 admin center → **Billing** → **Licenses**. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included).

2.2 Devices reporting into Intune

Description: Targeted devices are enrolled in Intune and checking in. **Why:** Reports only count enrolled, checked-in devices. **Verify:** Intune → **Devices** → **All devices**; run `EPRPT-Discovery.ps1`. **Implement:** Enable automatic enrollment; enroll/sync devices.

2.3 Reporting permissions (read-only Graph scopes)

Description: An operator/app with read scopes: `DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, `Group.Read.All`. **Why:** The discovery, validation, and report scripts read Intune data through Microsoft Graph. **Verify:** Run `EPRPT-Discovery.ps1` (it confirms it can sign in and read each data source). **Implement:** Sign in interactively with an Intune-read admin, or grant the app registration below.

2.4 Microsoft Graph export app registration

Description: An Entra app registration `Endpoint Reporting Export – Tenant – Prod` with application permission `DeviceManagementManagedDevices.Read.All` (and the read scopes above), admin-consented, used for unattended scheduled exports. **Why:** Scheduled, unattended Graph `exportJobs` need an app identity, not an interactive user. **Verify:** Entra → **App registrations** → the app

exists with consented permissions; run `EPRPT-Export-Scheduled.ps1` in its default dry single-run mode. **Implement:** Entra → **App registrations** → **New registration** → name it exactly as above → **API permissions** → **Add** → **Microsoft Graph** → **Application permissions** → add the read scopes → **Grant admin consent** → create a client secret or certificate stored as an environment secret (never in code).

2.5 Reporting-scope groups

Description: Two Entra security groups: `Endpoint Reporting Scope – Pilot – Stage` and `Endpoint Reporting Scope – Fleet – Prod`. **Why:** Pilot validation must be measured separately from the full fleet until the numbers reconcile. **Verify:** Entra → **Groups**; both exist. Run `EPRPT-Set-ReportingScopeTargeting.ps1` (read-mode). **Implement:** Run `EPRPT-Set-ReportingScopeTargeting.ps1` to create/populate them, or create manually (**Groups** → **New group** → **Security**).

2.6 A reporting destination

Description: A location to receive exports and host the dashboard (a secured file share/SharePoint library plus your dashboard tool, e.g. Power BI or Excel). **Why:** Scheduled exports and the consolidated KPI report need somewhere consistent to land. **Verify:** Confirm the export path is writable; confirm the dashboard tool can read it. **Implement:** Provision the share/library and point `EPRPT-Export-Scheduled.ps1 -OutputPath` at it.

2.7 (If applicable) Configuration Manager reporting point

Description: A working Configuration Manager **Reporting Services point** (SSRS) for any Configuration-Manager-managed devices. **Why:** Devices still managed by Configuration Manager are reported via SSRS, not Intune. **Verify:** Configuration Manager console → **Monitoring** → **Reporting** → **Reports** opens and renders a built-in software-update/compliance report. **Implement:** Install/repair the Reporting Services point role; confirm the SSRS URL resolves.

2.8 Defined KPI set agreed with stakeholders

Description: The five KPIs and their single source (see Overview) signed off by the report owner. **Why:** A KPI without an agreed definition and source is where "whose number is right" starts. **Verify:** The KPI definitions in `Endpoint KPI Report – Fleet – Prod` match this document. **Implement:** Adopt the KPI table from the Solution Overview; record it in the report definition.

Checklist

- Intune licensing assigned
- Devices enrolled and checking in — `EPRPT-Discovery.ps1`
- Read-only Graph scopes available — `EPRPT-Discovery.ps1`
- App registration `Endpoint Reporting Export – Tenant – Prod` consented
- Reporting-scope groups (Pilot / Fleet) created
- Reporting destination provisioned and writable

- (If applicable) Configuration Manager SSRS reporting point working
- KPI set defined and agreed

3. Implementation Guide — Endpoint Reporting & Dashboards

Follow these steps **in order**. Intune work is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`); Configuration Manager work is in the **Configuration Manager console**.

> ***Model:*** *Define the KPI set once, source each KPI from the right built-in report, automate the pull with a Graph export, then validate the exported numbers reconcile against the consoles on a* ***pilot*** *scope before distributing broadly. The repeatable way:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

Objects you will create, named with the `<Purpose> – <Scope> – <Stage>` convention (reuse verbatim):

- `Endpoint Reporting Scope – Pilot – Stage` (Entra group)
- `Endpoint Reporting Scope – Fleet – Prod` (Entra group)
- `Endpoint Reporting Export – Tenant – Prod` (app registration)
- `Endpoint KPI Report – Fleet – Prod` (report definition)
- `Endpoint KPI Distribution – Leadership – Prod` (distribution schedule)

Step 1 — Confirm readiness

1. Run `EPRPT-Discovery.ps1`. Confirm it signs in and reads each data source (managed devices, compliance, configuration, apps) and reports device counts. Resolve any source it cannot read before continuing.

Step 2 — Define the KPI set

1. Adopt the five KPIs and their single sources from the Solution Overview (patch compliance %, device compliance %, encryption coverage %, app deployment success %, device health %).
1. Record the definitions in the report definition `Endpoint KPI Report – Fleet – Prod` (a documented sheet/notes file that the KPI report script and dashboard both reference).

Step 3 — Create the reporting-scope groups

1. Run `EPRPT-Set-ReportingScopeTargeting.ps1` to create and populate `Endpoint Reporting Scope – Pilot – Stage` (a few representative devices) and `Endpoint Reporting Scope – Fleet – Prod` (the rest), or create them manually in Entra → **Groups** → **New group** → **Security** with those exact names.

Step 4 — Locate and confirm the built-in Intune reports

1. Intune → **Reports**. Open each source report and confirm it renders for your tenant:

• **Device compliance** (Reports → **Device compliance**) → device compliance %.

• **Windows updates** (Reports → **Windows updates**) → patch compliance %.

• **Endpoint security** → **Encryption report** (or Reports) → encryption coverage %.

• **Apps** → **Monitor** → **App install status** → app deployment success %.

• **Microsoft Defender** / **Device health** views → device health %.

1. Note that these are the on-screen, authoritative numbers you will reconcile the exports against.

Step 5 — Register the Graph export app

1. Entra → **App registrations** → **New registration** → name it `Endpoint Reporting Export – Tenant – Prod`.

2. **API permissions** → **Add** → **Microsoft Graph** → **Application permissions** → add

`DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, `Group.Read.All` → **Grant admin consent**.

1. Create a client secret (or upload a certificate). Store the tenant ID, client ID, and secret as

environment secrets for the export script — never in the script file.

Step 6 — Set up the scheduled export (dry run first)

1. Run `EPRPT-Export-Scheduled.ps1` with no schedule flag — it defaults to a **dry single run**:

it requests the Intune report `exportJobs`, polls until ready, downloads the CSV/JSON, and writes it to `-OutputPath` (your reporting destination).

1. Confirm the export files land in the reporting destination and open cleanly.

2. Only after the dry run succeeds, schedule it (Windows Task Scheduler / Azure Automation) on your

chosen cadence using the app registration credentials — this becomes `Endpoint KPI Distribution – Leadership – Prod`'s data feed.

Step 7 — (If applicable) Locate the Configuration Manager reports

1. Configuration Manager console → **Monitoring** → **Reporting** → **Reports**.

2. Open the built-in **Software Updates – A Compliance** and device-collection compliance reports;

confirm they render from the SSRS reporting point. These cover Configuration-Manager-managed devices and are folded into the consolidated view.

Step 8 — Build the consolidated KPI report

1. Run `EPRPT-Report.ps1` against the **pilot** scope first (`-Group "Endpoint Reporting Scope – Pilot – Stage"`).

It produces the consolidated endpoint KPI report (patch / compliance / encryption / app / health) from Graph, and a per-device CSV.

1. Point your dashboard tool (Power BI / Excel) at the report CSV + the scheduled export files to render the dashboard for `Endpoint KPI Report – Fleet – Prod`.

Step 9 — Validate the pilot (the gate)

1. Run `EPRPT-Validation.ps1` for the pilot scope. It reconciles the report/export counts against the live Intune source counts and flags any KPI whose numbers do not match within tolerance.
1. Spot-check each KPI in the Intune **Reports** portal (Step 4) against the report figure.
2. **Do not distribute broadly** until every KPI reconciles. This is the pilot gate.

Step 10 — Expand and distribute

1. Re-run `EPRPT-Report.ps1` against `Endpoint Reporting Scope – Fleet – Prod`.
2. Run `EPRPT-Validation.ps1` for the fleet scope and confirm reconciliation.
3. Turn on the recurring distribution `Endpoint KPI Distribution – Leadership – Prod` (the scheduled export from Step 6 plus dashboard refresh). Re-validate each cycle.

Order of operations

`Discovery → Define KPIs → Scope groups → Locate built-in reports → Register export app → Dry export → Configuration Manager reports → Build KPI report → Validate pilot (gate) → Expand + distribute`

5. Testing Guide — Endpoint Reporting & Dashboards

Run after the pilot is built (Implementation Step 8) and after each distribution cycle. The goal of testing here is **reconciliation** — the report/export numbers must equal the console numbers.

Test 1 — Data sources readable

Do: Run `EPRPT-Discovery.ps1`. **Expected:** It signs in and reads managed devices, compliance, configuration, and apps, and reports non-zero device counts for the scope.

Test 2 — Scope groups correct

Do: Run `EPRPT-Set-ReportingScopeTargeting.ps1` in read mode. **Expected:** `Endpoint Reporting Scope – Pilot – Stage` and `Endpoint Reporting Scope – Fleet – Prod` exist with the expected membership counts.

Test 3 — Export produces files

Do: Run `EPRPT-Export-Scheduled.ps1` (default dry single run) to the reporting destination. **Expected:** One or more CSV/JSON export files land in `-OutputPath` and open cleanly.

Test 4 — KPI report renders

Do: Run `EPRPT-Report.ps1 -Group "Endpoint Reporting Scope – Pilot – Stage"`. **Expected:** A consolidated report shows patch %, compliance %, encryption %, app success %, and health % plus a per-device CSV.

Test 5 — Numbers reconcile (the gate)

Do: Run `EPRPT-Validation.ps1` for the pilot scope; compare each KPI to the matching Intune **Reports** portal view (Device compliance, Windows updates, Encryption, App install status). **Expected:** Each KPI matches the console within the validation tolerance; no KPI flagged as mismatched.

Test 6 — Configuration Manager devices counted (if applicable)

Do: Open Configuration Manager console → **Monitoring** → **Reporting** → **Reports** and compare the software-update/compliance figures to the consolidated report's Configuration Manager portion. **Expected:** Configuration-Manager-managed devices are represented and consistent.

Test 7 — Distribution cycle

Do: Confirm the scheduled export `Endpoint KPI Distribution – Leadership – Prod` produced its latest files and the dashboard refreshed. **Expected:** Current-dated export files exist; the dashboard shows the latest KPI values.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 pass; every KPI reconciles for `Endpoint Reporting Scope – Pilot – Stage`
Fleet pass	Tests 1–7 pass for `Endpoint Reporting Scope – Fleet – Prod` before broad distribution

6. Troubleshooting Guide — Endpoint Reporting & Dashboards

Use `EPRPT-Discovery.ps1` to confirm data-source access, `EPRPT-Validation.ps1` to reconcile counts, and `EPRPT-Report.ps1` for the consolidated KPI view.

A. Cannot sign in / read a data source

Symptom	Cause	Remediation
Discovery fails to connect	Graph SDK missing or no consent	Install the Microsoft Graph SDK; sign in with an Intune-read admin; grant the read scopes
One source reads, another does not	Missing a specific read scope	Add the missing scope (`DeviceManagementApps.Read.All`, etc.) and re-consent
Unattended export 401/403	App registration not consented or wrong secret	Confirm admin consent on `Endpoint Reporting Export – Tenant – Prod`; refresh the client secret/certificate

B. Numbers don't reconcile (KPI mismatch)

Symptom	Cause	Remediation
Report % differs from portal	Different scope or filter	Confirm the report runs against the same group (`Endpoint Reporting Scope – …`) and the same OS/state filter as the portal view
Counts lower than expected	Stale devices not checked in	Devices report last-known state; let them sync, or document the "last seen" window in the KPI definition
Encryption % off	Encryption report still calculating	The encryption report can lag after enrollment; re-run after the device reports encryption status
App success % off	In-progress installs counted	Reconcile only completed install states; exclude pending/in-progress per the KPI definition

C. Export job problems

Symptom	Cause	Remediation
Export never completes	`exportJobs` still processing	The job is async; let `EPRPT-Export-Scheduled.ps1` poll to completion before downloading
Empty/partial export	Wrong report name or filter	Confirm the requested report name and filter; large tenants page — let the script follow `@odata.nextLink`

Symptom	Cause	Remediation
Files don't land	Output path not writable	Point `-OutputPath` at a writable reporting destination; check share/library permissions

D. Configuration Manager reports

• Reports won't render in **Monitoring** → **Reporting** → the **Reporting Services point** (SSRS) is down or mis-URL'd; confirm the SSRS reporting point role and that the report URL resolves.

• Configuration Manager devices missing from the consolidated view → confirm those collections report and that you are reading the SSRS software-update/compliance reports, not only Intune.

E. Scope-group issues

• Pilot and fleet numbers blur together → confirm `EPRPT-Set-ReportingScopeTargeting.ps1` populated the correct, separate groups and that the report `-Group` parameter targets the intended scope.

F. Diagnostics quick reference

```
Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome   # confirm read access
Invoke-MgGraphRequest -Method GET -Uri '/v1.0/deviceManagement/managedDevices?$top=1'   # smoke test
Invoke-MgGraphRequest -Method GET -Uri '/beta/deviceManagement/reports/exportJobs'      # list export jobs
```

G. When to escalate

If discovery confirms read access, scopes are consented, devices are synced, and a KPI still will not reconcile after matching scope and filter to the portal, capture the script output and the portal view side by side and open a Microsoft support case for the affected report.

7. Best Practices — Endpoint Reporting & Dashboards

• **Define the KPI and its single source first.** The most important practice: one authoritative source per KPI means the same question always returns the same number. Disagreement starts when two people pull the same KPI two different ways.

• **Validate before you distribute.** Reconcile every KPI against the console on the pilot scope before broad distribution — the pilot gate is what makes the numbers trustworthy.

• **Automate the pull, don't hand-build it.** Use the Graph `exportJobs` so the figures are reproduced by the same code every cycle, not re-derived in a spreadsheet.

• **Name everything consistently.** Use `<Purpose> – <Scope> – <Stage>` for groups, the export app,

the report definition, and the distribution schedule, and reuse the names verbatim so the objects are findable and auditable.

• **Keep pilot and fleet scopes separate** until the pilot reconciles; never report a number you have not validated against its source.

• **Store secrets outside the code.** The export app's client secret/certificate lives in an environment secret or secrets vault — never in the script or a config committed to source.

• **Use read-only access for reporting.** Discovery, validation, and the KPI report only need read scopes; do not grant write permissions to a reporting identity.

• **Cover the whole fleet.** If any devices are still managed by Configuration Manager, include the SSRS reports so the consolidated view is complete, not just the Intune slice.

• **Timestamp and archive exports.** Keep dated export files so an auditor can see the exact data behind a past report.

• **Document the cadence and tolerances.** Record the distribution schedule and the reconciliation tolerance used in validation, so the process is repeatable by the next operator.

8. Deliverables Summary — Endpoint Reporting & Dashboards

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Intune/Configuration Manager data sources, the KPI set, Graph export, the reporting model, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing, and the named objects
Global Requirements (Infrastructure)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: KPIs, built-in reports, Graph export, dashboard, distribution, validation
5. Testing Guide	Reconciliation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`EPRPT-Discovery.ps1`	Reporting-readiness: confirms it can sign in and read each Intune data source (read-only)
`EPRPT-Validation.ps1`	Reconciles report/export counts against the live Intune source counts (read-only)
`EPRPT-Report.ps1`	Consolidated endpoint KPI report — patch / compliance / encryption / app / health (read-only)
`EPRPT-Export-Scheduled.ps1`	Microsoft Graph `exportJobs` helper; defaults to a dry single run
`EPRPT-Set-ReportingScopeTargeting.ps1`	Builds/populates the reporting-scope Entra groups

Named objects (reuse verbatim)

- App registration: `Endpoint Reporting Export – Tenant – Prod`
- Reporting-scope groups: `Endpoint Reporting Scope – Pilot – Stage`, `Endpoint Reporting Scope – Fleet – Prod`
- Report definition: `Endpoint KPI Report – Fleet – Prod`
- Distribution schedule: `Endpoint KPI Distribution – Leadership – Prod`

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a reconciliation gate before broad distribution.

Outcome

Trustworthy, repeatable endpoint reporting — patch, compliance, encryption, app deployment, and device health across Intune and Configuration Manager — with the same numbers every time, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EPRPT-Discovery.ps1

Purpose: Confirms the data sources and permissions needed for endpoint reporting are reachable: signs in to Microsoft Graph and reads managed devices, compliance, device configuration, and apps, reporting a count for each so you know the reporting layer can be built. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EPRPT-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and read scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS A line per data source with a count, and an overall READY / NOT READY result. .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All','DeviceManagementConfiguration.Read.All','DeviceManagementApps.Read.All','Group.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\EPRPT-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and read scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS A line per data source with a count, and an overall READY / NOT READY result. .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All','DeviceManagementConfiguration.Read.All','DeviceManagementApps.Read.All','Group.Read.All' -NoWelcome

EPRPT-Export-Scheduled.ps1

Purpose: Requests an Intune report export through the Microsoft Graph deviceManagement reports exportJobs endpoint, polls until the job completes, and downloads the resulting file (CSV inside a ZIP) to a reporting destination. This is the automatable pull that lets the same KPI data be reproduced on a schedule. DEFAULTS TO A DRY SINGLE RUN — it performs one export and exits; it does NOT install a recurring schedule (do that in Task Scheduler / Azure Automation once the dry run succeeds). HOW TO RUN Dry single run (default): powershell -ExecutionPolicy Bypass -File .\EPRPT-Export-Scheduled.ps1 -OutputPath C:\Reporting\export Choose a report (default DeviceCompliance): ... -ReportName Devices -OutputPath C:\Reporting\export Unattended (app registration), pass app credentials via environment secrets, not literals. Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All (read). EXPECTED RESULTS One downloaded export file in -OutputPath and a SUCCESS line; no recurring schedule is created. .NOTES Workplace Marketplace — Infrastructure. Reads/exports Intune report data via Microsoft Graph. Common report names: DeviceCompliance, Devices, DeviceInstallStatusByApp, BitLockerReport, FeatureUpdateDeviceState (confirm the current list in your tenant before scheduling). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$OutputPath, [string]$ReportName = 'DeviceCompliance', [int]$TimeoutSeconds = 300) if (-not (Test-Path $OutputPath)) { New-Item -ItemType Directory -Path

$OutputPath -Force | Out-Null } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome

How to run: Dry single run (default): powershell -ExecutionPolicy Bypass -File .\EPRPT-Export-Scheduled.ps1 -OutputPath C:\Reporting\export Choose a report (default DeviceCompliance): ... -ReportName Devices -OutputPath C:\Reporting\export Unattended (app registration), pass app credentials via environment secrets, not literals. Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All (read). EXPECTED RESULTS One downloaded export file in -OutputPath and a SUCCESS line; no recurring schedule is created. .NOTES Workplace Marketplace — Infrastructure. Reads/exports Intune report data via Microsoft Graph. Common report names: DeviceCompliance, Devices, DeviceInstallStatusByApp, BitLockerReport, FeatureUpdateDeviceState (confirm the current list in your tenant before scheduling). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$OutputPath, [string]$ReportName = 'DeviceCompliance', [int]$TimeoutSeconds = 300) if (-not (Test-Path $OutputPath)) { New-Item -ItemType Directory -Path $OutputPath -Force | Out-Null } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome

EPRPT-Report.ps1

Purpose: Produces the consolidated endpoint KPI report from Intune data via Microsoft Graph: patch compliance %, device compliance %, encryption coverage %, app deployment success %, and device health %, optionally scoped to a reporting-scope group. Writes a summary to the console and a per-device CSV that EPRPT-Validation.ps1 can reconcile. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EPRPT-Report.ps1 -CsvPath C:\Temp\eprpt-report.csv [-Group "Endpoint Reporting Scope - Pilot - Stage"] Requires the Microsoft Graph SDK and read scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS The five endpoint KPIs as percentages and a per-device CSV (device, OS, compliant, encrypted, health, last sync). .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [string]$Group) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All','DeviceManagementApps.Read.All','Group.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\EPRPT-Report.ps1 -CsvPath C:\Temp\eprpt-report.csv [-Group "Endpoint Reporting Scope - Pilot - Stage"] Requires the Microsoft Graph SDK and read scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS The five endpoint KPIs as percentages and a per-device CSV (device, OS, compliant, encrypted, health, last sync). .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [string]$Group) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes

'DeviceManagementManagedDevices.Read.All','DeviceManagementApps.Read.All','Group.Read.All' -NoWelcome

EPRPT-Set-ReportingScopeTargeting.ps1

Purpose: Creates (or, in read mode, just reports on) the two reporting-scope groups used to target reporting and exports at a pilot population first and the whole fleet second: 'Endpoint Reporting Scope - Pilot - Stage' 'Endpoint Reporting Scope - Fleet - Prod' Optionally adds device object IDs to the pilot group from a supplied list. HOW TO RUN Report only (default, read-only): powershell -ExecutionPolicy Bypass -File .\EPRPT-Set-ReportingScopeTargeting.ps1 Create the groups if missing: ... -Create Add pilot members (Entra device object IDs): ... -Create -PilotDeviceObjectIds id1,id2,id3 Requires the Microsoft Graph SDK. Read mode: Group.Read.All. Create/add: Group.ReadWrite.All. EXPECTED RESULTS Read mode: existence + membership count for each group. Create mode: the groups created if missing and pilot members added; a final summary. .NOTES Workplace Marketplace — Infrastructure. Read-only unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create, [string[]]$PilotDeviceObjectIds = @()) $pilot = 'Endpoint Reporting Scope - Pilot - Stage' $fleet = 'Endpoint Reporting Scope - Fleet - Prod'

How to run: Report only (default, read-only): powershell -ExecutionPolicy Bypass -File .\EPRPT-Set-ReportingScopeTargeting.ps1 Create the groups if missing: ... -Create Add pilot members (Entra device object IDs): ... -Create -PilotDeviceObjectIds id1,id2,id3 Requires the Microsoft Graph SDK. Read mode: Group.Read.All. Create/add: Group.ReadWrite.All. EXPECTED RESULTS Read mode: existence + membership count for each group. Create mode: the groups created if missing and pilot members added; a final summary. .NOTES Workplace Marketplace — Infrastructure. Read-only unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create, [string[]]$PilotDeviceObjectIds = @()) $pilot = 'Endpoint Reporting Scope - Pilot - Stage' $fleet = 'Endpoint Reporting Scope - Fleet - Prod'

EPRPT-Validation.ps1

Purpose: Recomputes the in-scope device totals straight from Microsoft Graph (managed devices, compliant devices, encryption-on devices) and compares them to a previously generated report CSV (from EPRPT-Report.ps1). Flags any KPI whose numbers do not match within a tolerance, so you only distribute numbers that reconcile (the pilot gate). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EPRPT-Validation.ps1 -ReportCsv C:\Temp\eprpt-report.csv [-TolerancePct 1] Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-KPI comparison (source vs report) and an overall RECONCILED / MISMATCH result. .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ReportCsv, [int]$TolerancePct = 1) if (-not (Test-Path $ReportCsv)) { throw "Report CSV not found: $ReportCsv" } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\EPRPT-Validation.ps1 -ReportCsv C:\Temp\eprpt-report.csv [-TolerancePct 1] Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-KPI comparison (source vs report) and an overall RECONCILED / MISMATCH result. .NOTES Workplace Marketplace — Infrastructure. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ReportCsv, [int]$TolerancePct = 1) if (-not (Test-Path $ReportCsv)) { throw "Report CSV not found: $ReportCsv" } if (-not (Get-Module -ListAvailable -Name

```
Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force
-AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph
-Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome
```